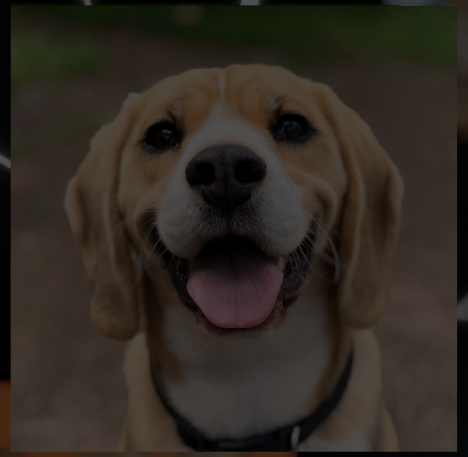
Practice 01:

Training a game-related classifier



MyeongAh Cho

Kyung Hee University

Artificial Intelligence and Game Programming

Contents

- Mini Tutorial for Practice
- About Assignment

1. Import libraries

- Import essential libraries for **data processing, visualization, and deep learning**
- Prepare tools for **dataset generation, model training, and evaluation**
- Set random seeds for **reproducibility**

```python
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs
from sklearn.metrics import accuracy_score

np.random.seed(42)
torch.manual_seed(42)
```

1. Import libraries

- **Numerical & Visualization**
  - numpy: numerical operations and array handling
  - matplotlib.pyplot: data visualization (e.g., plots)
- **PyTorch (Deep Learning)**
  - torch: core tensor operations
  - torch.nn: neural network layers and models
  - torch.optim: optimization algorithms (e.g., Adam)
  - TensorDataset, DataLoader: data handling and batching

1. Import libraries

- **Scikit-learn (Utility Tools)**
  - train_test_split: split dataset into training and testing sets
  - StandardScaler: normalize input features
  - make_blobs: generate synthetic dataset for classification
  - accuracy_score: evaluate model performance
- **Reproducibility**
  - Random data generation is consistent
  - Model initialization is reproducible → Experimental results can be replicated

2. Create a Very Simple Dataset

- We will generate a small 2D classification dataset.
  - Each sample has:
    - $x1, x2$
  - Each label is:
    - 0 or 1
- Generate a **simple classification dataset**
- Each sample has **2 input features (x1, x2)**
- The target is a **binary label (0 or 1)**

```
X, y = make_blobs(
    n_samples=500,
    centers=2,
    n_features=2,
    cluster_std=2.0,
    random_state=42
)

print('X shape:', X.shape)
print('y shape:', y.shape)
```

2. Create a Very Simple Dataset

- **Key Parameters**
  - n_samples=500 → total number of data points
  - centers=2 → number of classes (binary classification)
  - n_features=2 → number of input dimension (x1, x2)
  - cluster_std=2.0 → spread of each cluster (controls difficulty)
  - random_state=42 → ensures reproducibility
- **Output**
  - X: input features → shape (500, 2)
  - y: labels → shape (500,)

```python
X, y = make_blobs(
    n_samples=500,
    centers=2,
    n_features=2,
    cluster_std=2.0,
    random_state=42
)

print('X shape:', X.shape)
print('y shape:', y.shape)
```

2. Create a Very Simple Dataset

- **Key Parameters**
  - n_samples=500 → total number of data points
  - centers=2 → number of classes (binary classification)
  - n_features=2 → number of input dimension (x1, x2)
  - cluster_std=2.0 → spread of each cluster (controls difficulty)
  - random_state=42 → ensures reproducibility
- **Output**
  - X: input features → shape (500, 2)
  - y: labels → shape (500,)

```python
X, y = make_blobs(
    n_samples=500,
    centers=2,
    n_features=2,
    cluster_std=2.0,
    random_state=42
)

print('X shape:', X.shape)
print('y shape:', y.shape)
```

2. Create a Very Simple Dataset

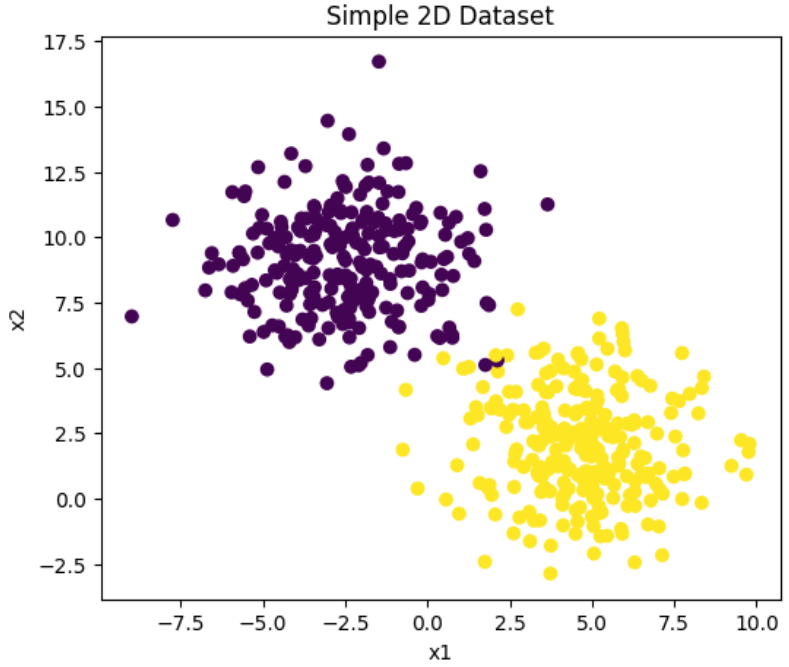
- **Visualizing**
  - Plot the dataset in a **2D space**
  - Visually understand how data points are distributed
  - Identify how classes are separated

```
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title('Simple 2D Dataset')
plt.xlabel('x1')
plt.ylabel('x2')
plt.show()
```

2. Create a Very Simple Dataset

- plt.figure(figsize=(6, 5))
  - → Create a new figure with a fixed size
  - → Controls the width and height of the plot
- plt.scatter(X[:, 0], X[:, 1], c=y)
  - → Draw a scatter plot
  - → X[:, 0]: values of feature **x1** (x-axis)
  - → X[:, 1]: values of feature **x2** (y-axis)
  - → c=y: color each point based on its label

```
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title('Simple 2D Dataset')
plt.xlabel('x1')
plt.ylabel('x2')
plt.show()
```



3. Split into Train and Test Data

- X, y

  → Input data and labels

- test_size=0.2

  → 20% of the data is used as the test set

  → 80% is used for training

- stratify=y

  → Keeps the **same class distribution** in both train and test sets

  → Important for balanced evaluation

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print('Train:', X_train.shape, y_train.shape)
print('Test :', X_test.shape, y_test.shape)
```

4. Normalize the Input Data

- Normalize input features to a similar scale
  - Standardizes data to **mean = 0, standard deviation = 1**
- Improve **training stability and convergence speed**
- Prevent features with large values from dominating the model

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(X_train[:3])
```

4. Normalize the Input Data

- scaler.fit_transform(X_train)

  → Compute mean and standard deviation from **training data**

  → Apply normalization to training data

- scaler.transform(X_test)

  → Apply the **same scaling parameters** to test data

  → Do NOT recompute statistics from test data

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(X_train[:3])
```

5. Convert NumPy Arrays to PyTorch Tensors

- PyTorch models work with **tensors**, not NumPy arrays.
  - input data: float32
  - class labels: long

```
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)

X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

print(X_train_tensor.shape)
print(y_train_tensor.shape)
```

6. TensorDataset and DataLoader

- Why do we use them?
  - TensorDataset: combines input and label tensors into one dataset
  - DataLoader: sends data to the model in small batches
- This is useful because we usually do **mini-batch training**, not full-data training at once.

```
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print('Number of train batches:', len(train_loader))
print('Number of test batches :', len(test_loader))
```

7. Build a Simple Neural Network

- We create a small MLP:
  - input size = 2
  - hidden layer = 8 neurons
  - output size = 2 classes
- For binary classification with 2 output classes, we can use:
  - final linear layer with 2 outputs
  - CrossEntropyLoss

```python
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 8),
            nn.ReLU(),
            nn.Linear(8, 2)
        )

    def forward(self, x):
        return self.net(x)

model = SimpleMLP()
print(model)
```

7. Build a Simple Neural Network

- **nn.Module:** defines a custom neural network class
  - Inherits from nn.Module, the base class for all PyTorch models
  - Enables automatic handling of parameters and gradients
- **nn.Linear(in_features, out_features)**
  - in_features: Number of input features
  - out_features: Number of output neurons
- **forward(self, x)**
  - x → input tensor
  - Returns model output (logits)

```python
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 8),
            nn.ReLU(),
            nn.Linear(8, 2)
        )

    def forward(self, x):
        return self.net(x)

model = SimpleMLP()
print(model)
```

8. Define Loss Function and Optimizer

- Loss function
  - CrossEntropyLoss compares model outputs and true class labels.
- Optimizer
  - Adam updates the model parameters using gradients.

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

9. Train Function

```python
def train_model(model, train_loader, criterion, optimizer, epochs=30):
    train_losses = []
    train_accuracies = []

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        all_preds = []
        all_labels = []

        for xb, yb in train_loader:
            optimizer.zero_grad()

            outputs = model(xb)
            loss = criterion(outputs, yb)

            loss.backward()
            optimizer.step()

            total_loss += loss.item() * xb.size(0)

            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.detach().numpy())
            all_labels.extend(yb.detach().numpy())

        avg_loss = total_loss / len(train_loader.dataset)
        acc = accuracy_score(all_labels, all_preds)

        train_losses.append(avg_loss)
        train_accuracies.append(acc)

        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f'Epoch {epoch+1:02d} | Loss: {avg_loss:.4f} | Accuracy: {acc:.4f}')

    return train_losses, train_accuracies
```

10. Run Training

```
train_losses, train_accuracies = train_model(
    model,
    train_loader,
    criterion,
    optimizer,
    epochs=30
)
```

```
Epoch 01 | Loss: 0.6771 | Accuracy: 0.6000
Epoch 0  데이터 검사 도구  0.0747 | Accuracy: 0.9900
Epoch 10 | Loss: 0.0306 | Accuracy: 0.9900
Epoch 15 | Loss: 0.0268 | Accuracy: 0.9900
Epoch 20 | Loss: 0.0253 | Accuracy: 0.9900
Epoch 25 | Loss: 0.0234 | Accuracy: 0.9900
Epoch 30 | Loss: 0.0226 | Accuracy: 0.9875
```

11. Test Function

- **model.eval()**
  - Sets the model to **evaluation mode**
  - Disables training-specific behaviors such as:
    - Dropout
- **torch.no_grad()**
  - Disables gradient computation
  - Reduces memory usage
  - Speeds up inference
  - Required during testing (no backpropagation)

```python
def test_model(model, test_loader, criterion):
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for xb, yb in test_loader:
            outputs = model(xb)
            loss = criterion(outputs, yb)

            total_loss += loss.item() * xb.size(0)

            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.numpy())
            all_labels.extend(yb.numpy())

    avg_loss = total_loss / len(test_loader.dataset)
    acc = accuracy_score(all_labels, all_preds)
    return avg_loss, acc, all_preds, all_labels
```

11. Test Function

- **accuracy_score(all_labels, all_preds)**
  - Computes classification accuracy
  - From sklearn.metrics
  - Formula:
    - (number of correct predictions) / (total samples)

```python
def test_model(model, test_loader, criterion):
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for xb, yb in test_loader:
            outputs = model(xb)
            loss = criterion(outputs, yb)

            total_loss += loss.item() * xb.size(0)

            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.numpy())
            all_labels.extend(yb.numpy())

    avg_loss = total_loss / len(test_loader.dataset)
    acc = accuracy_score(all_labels, all_preds)
    return avg_loss, acc, all_preds, all_labels
```

11. Test Function

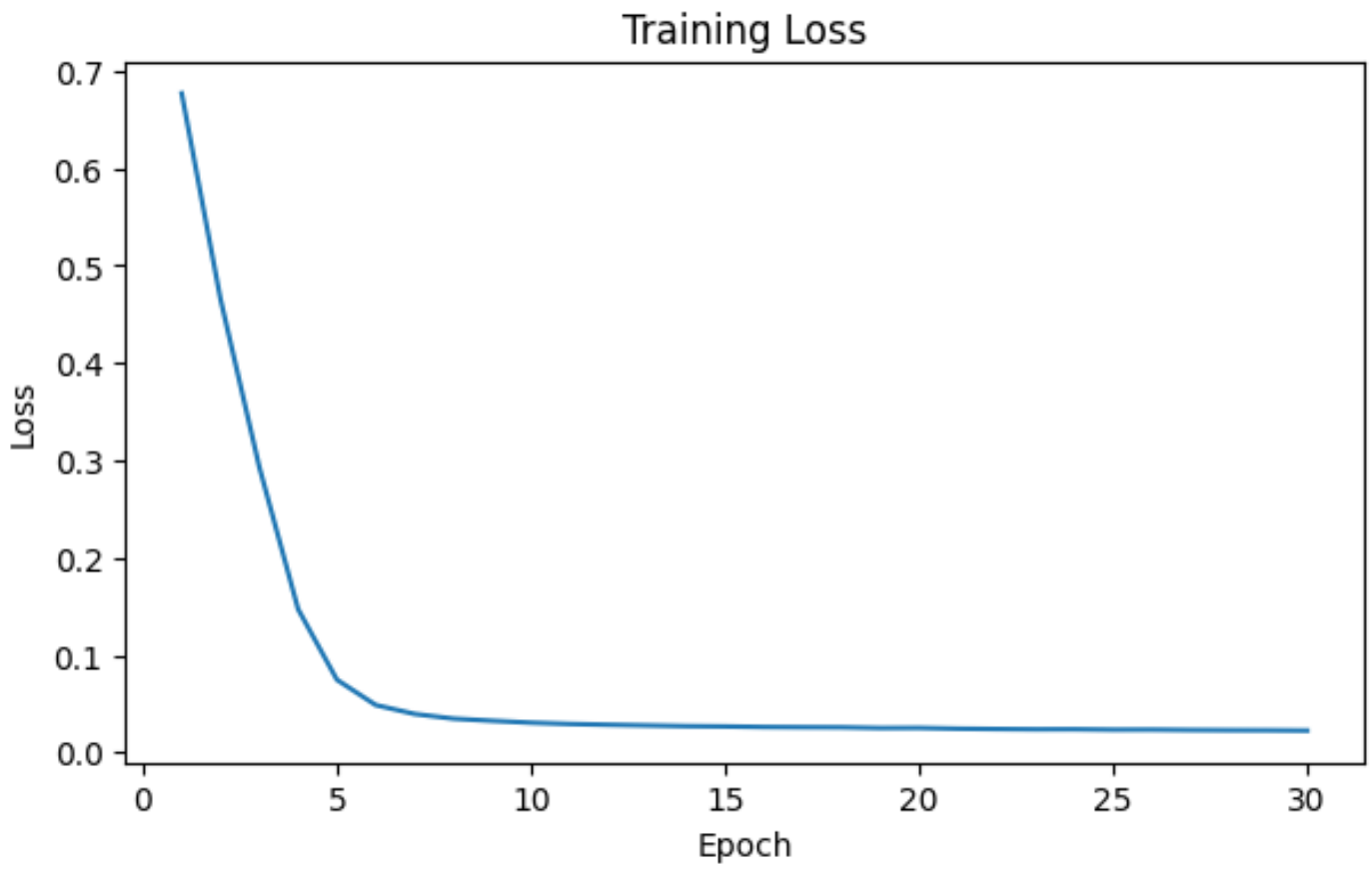
```
test_loss, test_acc, test_preds, test_labels = test_model(model, test_loader, criterion)
print('Test Loss    :', round(test_loss, 4))
print('Test Accuracy:', round(test_acc, 4))
```

```
Test Loss    : 0.0035
Test Accuracy: 1.0
```

12. Plot Training Results

```
epochs = range(1, len(train_losses) + 1)

plt.figure(figsize=(7, 4))
plt.plot(epochs, train_losses)
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```



Optional Simple Exercises

1. Change the hidden layer size from 8 to 16.
2. Change the learning rate from 0.01 to 0.001.
3. Change the batch size from 32 to 64.
4. Increase the number of epochs.
5. Add one more hidden layer.

- These are good quick experiments for understanding the library workflow.

Practice

- Goal: Design a **game-related classification problem**, train a neural network model, and analyze how model structure and regularization affect performance.
- Your notebook must clearly show:
  - your own problem design,
  - your own experiment setup,
  - your own result analysis.

Practice

- Requirements:
  - Your notebook must include all of the following:
    - **Game-related problem definition**
    - **Dataset generation or construction**
    - **At least one neural network model (MLP)**
    - **Training process with loss visualization**
    - **Model comparison**
      - shallow vs deeper model
      - no regularization vs regularization
    - **Analysis of overfitting / generalization**
    - **Final discussion and conclusion**

Practice

- *Example* Problem Ideas
  - NPC action classification: attack / retreat / idle
  - Danger zone classification from player position and state
  - Item usage decision prediction
  - Enemy threat level classification
  - Boss phase prediction from HP / distance / cooldown / nearby allies

- (1) You may also create your own problem, or (2) you can modify this settings

Practice

- **NPC Action Classification**
  - Input features:
    - distance_to_enemy
    - health
    - ammo
    - enemy_visible
    - cover_available
  - Output classes:
    - 0: retreat
    - 1: idle
    - 2: attack

```python
def generate_npc_dataset(n_samples=1200, noise_ratio=0.08):
    rng = np.random.default_rng(42)
    distance = rng.uniform(0, 100, n_samples)
    health = rng.uniform(0, 100, n_samples)
    ammo = rng.integers(0, 31, n_samples)
    enemy_visible = rng.integers(0, 2, n_samples)
    cover_available = rng.integers(0, 2, n_samples)

    X = np.column_stack([distance, health, ammo, enemy_visible, cover_available])
    y = []

    for d, h, a, v, c in X:
        if (h < 30 and d < 40 and v == 1) or (h < 25 and c == 0):
            label = 0  # retreat
        elif v == 1 and a >= 8 and h >= 35 and d < 55:
            label = 2  # attack
        else:
            label = 1  # idle
        y.append(label)

    y = np.array(y)

    # Inject small label noise to make the task more realistic
    n_noise = int(n_samples * noise_ratio)
    noise_idx = rng.choice(n_samples, size=n_noise, replace=False)
    y[noise_idx] = rng.integers(0, 3, size=n_noise)

    return X, y

X, y = generate_npc_dataset()
X.shape, y.shape
```

Practice

Required Analysis Questions

1. What is your game AI problem, and why did you design it this way?
2. Which features were most important in your task?
3. How did the shallow and deeper models differ?
4. Did you observe overfitting? How can you tell?
5. What effect did regularization have?
6. If you improved this project further, what would you change?

Practice

- Submission
  - Submit **one Jupyter notebook** that runs from top to bottom.
  - Your notebook should be understandable without oral explanation.
  - Submit via E-campus ~ **Apr. 6 (Wed.) 23:59**
  - You may write the content in Korean

Practice

- Bonus (20%)
  - You may implement additional Optional Extensions
  - You may also choose to present your work
  - Presentation
    - Duration: 10 minutes
    - Date: Apr. 7 (Thu.)
    - Contents
      - **Game-related problem definition**
      - **Analysis of overfitting / generalization**
      - **Final discussion and conclusion**

13. Optional Extensions (Bonus)

Try one or more of the following:

- Add **another model** for a three-way comparison
- Change the dataset difficulty and compare results
- Use **dropout**, **L2 regularization**, or **early stopping**
- Visualize decision boundaries for 2 selected features
- Create a simple interactive demo using widgets or sliders
- Analyze failure cases by printing misclassified examples

Thank you

End of Introduction

Questions?